# Sonne: Automated Installation Guide

Local AI assistant with Discord and a team website

This guide configures Discord and installs Sonne on an existing Linux system. The installer handles Docker, the model download and the Prompt Studio website. Team members sign in to the website with Discord and configure shared project information and their own assistant profiles. Linux commands, browser actions and Discord messages are identified separately throughout.

**Before you begin**

| | |
|---|---|
| System | Ubuntu 22.04 or 24.04 LTS, or Debian 13; 64-bit Intel/AMD (amd64) or ARM64. |
| Resources | Qwen: approximately 8 GiB RAM and 15 GiB free disk. Gemma: approximately 16 GiB RAM and 20 GiB free disk. Internet access is required. |
| Access | A regular Linux account with `sudo` privileges; a Discord account with permission to add a bot and create a text channel on the chosen server. |

Keep a private note of the bot token, OAuth2 Client Secret and the four types of ID collected below. Both secrets are credentials; do not include them in shared documents, screenshots or support messages. Use your own IDs; numbers in the figures are examples. Only the administrator enters the secrets; team members use their own Discord accounts to sign in.

## 1. Create the Discord application

Open the Discord Developer Portal and sign in. Select **Create / New Application**, enter **Sonne**, accept the displayed terms and create the application. Under **General Information**, copy and save the **Application ID**.

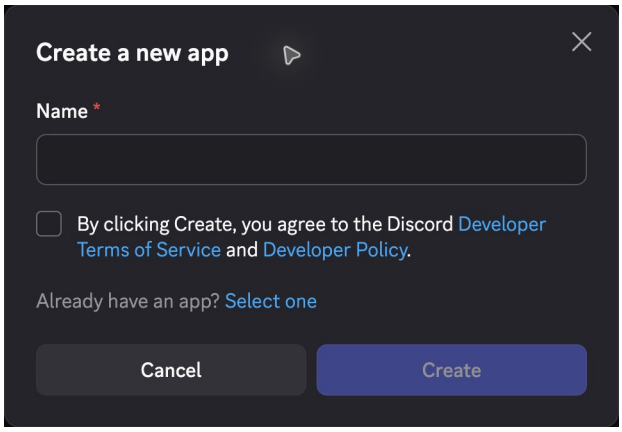



Figure 1. Create the application.

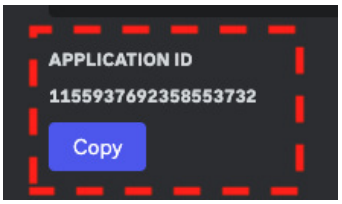



Figure 2. Copy your Application ID.




Figure 3. The token control on the Bot page.

## 2. Save the bot token and enable message content

Select **Bot** in the application's sidebar and copy the token. If no token is available to copy, select **Reset Token** and complete the confirmation. Save the new token immediately; resetting it invalidates the previous one.

Enable **Message Content Intent**. Leave **Presence Intent** and **Server Members Intent** off. Select **Save Changes** if prompted.

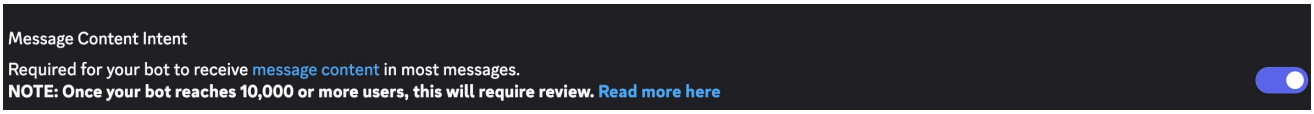



Figure 4. Message Content Intent enabled.

**Check:** You have saved the Application ID and bot token.

# Configure Discord access

## 3. Invite the bot to your server

Open **OAuth2 / URL Generator** in the Developer Portal. Select `bot` and `applications.commands` under **Scopes**.

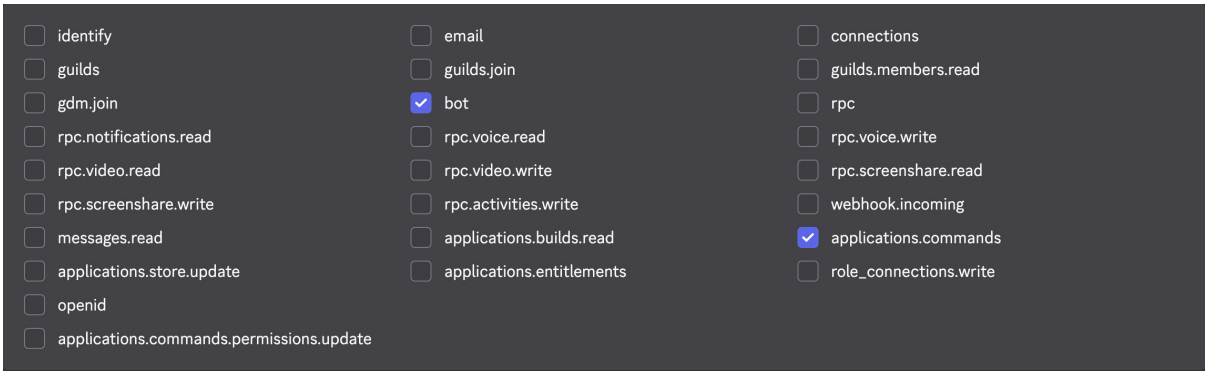



Figure 5. The two required scopes selected.

Under **Bot Permissions**, select **View Channels, Send Messages, Read Message History, Embed Links** and **Attach Files**. Leave **Administrator** unchecked. Open the **Generated URL** in your browser, choose your server and authorize the bot.

In Discord, select + beside **Text Channels**, create a text channel named `sonne` and allow the bot and intended users to view it and send messages.

## 4. Copy the server, channel and user IDs

In Discord, open **User Settings** (gear beside your username), then **Advanced**, and enable **Developer Mode**.

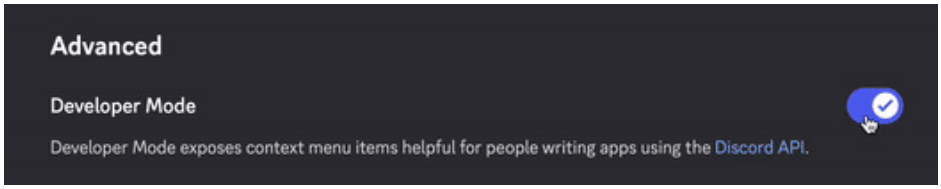



Figure 6. Developer Mode enables the Copy ID menu options.

Right-click the **server icon**: **Copy Server ID**; the **sonne channel**: **Copy Channel ID**; your name in a message or member list: **Copy User ID**. **Save each ID in a text file or note, labeled Server ID, Channel ID or User ID, for use during installation.** Repeat for additional permitted users; include yourself for testing.

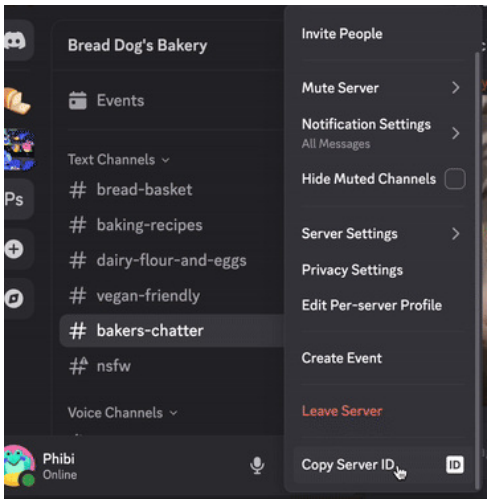



Figure 7. Copy Server ID.

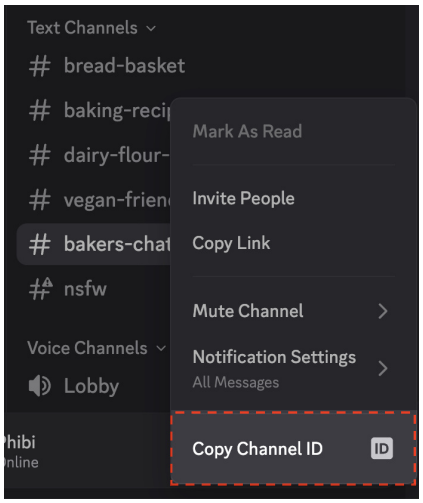



Figure 8. Copy Channel ID.

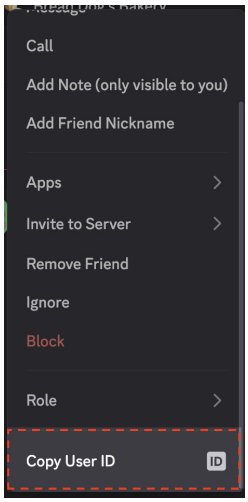



Figure 9. Copy User ID.

**Check:** Your note contains the token, Application ID, Server ID, Channel ID and at least one permitted User ID.

# Prepare Discord web sign-in

## 5. Save the OAuth2 Client Secret and register the redirect

Return to the Discord Developer Portal and select the **same Sonne application** used for the bot. Open **OAuth2** and its **General** settings, if shown.

1. Find **Client Secret**. Reveal or copy the existing secret if the portal offers that option. If no secret can be retrieved, use the portal's **Reset Secret** control and complete the confirmation. A reset invalidates the previous secret for any installation already using it.
2. Save the value privately as **Discord OAuth2 Client Secret**. This is separate from the **Bot token** and the numeric **Application ID**. The installer uses the Application ID as the OAuth2 Client ID.
3. Under **Redirects**, select **Add Redirect** and enter exactly:

```
http://localhost:9080/auth/discord/callback
```

Select **Save Changes**. Keep the `/auth/discord/callback` path; do not enter only the website's home address.

The default **Prompt Studio URL** is `http://localhost:9080`. Use the same address when the installer asks for the website URL and when opening it in your browser. `localhost` and `127.0.0.1` are distinct OAuth redirect addresses. If you choose another web address, register that exact address followed by `/auth/discord/callback` instead.

Sonne requests only Discord's `identify` scope for web sign-in. You do not need to generate another bot invitation URL for this login. See Discord's OAuth2 documentation.

**Check:** Your private note contains both secrets and all required IDs. The OAuth2 redirect is saved in the same Discord application.

## 6. Open a Linux terminal

On the target Linux system, open **Terminal** from the applications menu. If Linux runs in a virtual machine or on another computer, use a terminal in that Linux environment. Sign in as your regular user.

Run each command below on its own line and press **Enter**. Wait for the command prompt to return before entering the next command. To paste in a typical Linux terminal, use **Ctrl+Shift+V** or right-click and select **Paste**. When `sudo` requests a password, enter your **Linux login password** and press Enter. No characters or asterisks appear while you type.

# Install Sonne on Linux

## 7. Download and run the installer

The download folder `~/sonne-installer` must not already exist. The `~` symbol means your Linux home folder.

```
sudo apt-get update
sudo apt-get install -y git
git clone https://github.com/river-team-2026/Sonne.git ~/sonne-installer
cd ~/sonne-installer
bash scripts/install-sonne.sh
```

Run the script exactly as shown. First choose a model: **1 for Qwen3.5 2B** (default; about 1.9 GB download), or **2 for Gemma 4 E2B IT QAT** (about 4.3 GB download). Use the RAM requirements on page 1. The script requests administrative access when needed. It collects both secrets, all Discord IDs and the web address in one initial configuration. At each prompt, paste the corresponding value and press Enter:

| Installer prompt | Value to enter |
|---|---|
| Discord bot token | Token saved in step 2; input is hidden. |
| Discord OAuth2 Client Secret | Secret saved in step 5 from the same application; input is hidden. Do not enter the bot token here. |
| Application ID | Application ID from General Information, step 1. |
| Server ID | ID of the server to which you invited the bot. |
| Channel ID | ID of the `sonne` text channel. |
| Permitted User IDs | Your User ID and any additional permitted User IDs, separated by commas with no spaces. For one user, enter only that user's ID. |
| Prompt Studio URL | Press Enter to accept `http://localhost:9080`. Use the address whose callback you registered in step 5. |

Downloads and initial model loading may take several minutes. Keep the terminal open. The installer creates the running installation in `~/agents/sonne`; this is separate from the download folder. Later runs keep the saved model. To change it, rerun with `-model qwen` or `-model gemma`; the bot is paused during setup. User files, profiles and credentials are retained. The installer reuses saved values on later runs, so you do not need to enter the secrets again.

**Check:** The installer reports both messages below, without an error, before returning to the command prompt:

```
Local model routing and Discord probe passed. Verify a real mention and file/image tasks next.
Installed in /home/<your-user>/agents/sonne
```

The installation path contains your actual Linux home folder, which may differ from this example. At the end, the installer also prints **Prompt Studio** and **Discord OAuth2 redirect**. Check that the redirect matches the one saved in the Developer Portal. No separate web installation command is required.

If the installer reports *Using sudo for Docker now*, sign out of Linux completely and sign back in, or reconnect your SSH session, before running the Docker commands on the next page. Closing the terminal alone is insufficient. If installation stops, follow the recovery steps on the last page.

# Open the Sonne website

## 8. Sign in to Prompt Studio with Discord

In a browser on the Linux host, open `http://localhost:9080`. Select **Sign in with Discord**, choose a permitted Discord account and authorize access. Use an account whose numeric User ID you entered during setup.

**Check:** The workspace opens with your Discord account and its personal profile.

If Linux runs in a virtual machine or on another computer, `localhost` in your browser refers to the computer running that browser. Open a browser inside Linux, or run this command in a terminal on your own computer:

```
ssh -N -L 9080:127.0.0.1:9080 your-user@your-linux-host
```

Replace `your-user` and `your-linux-host` with the Linux login and host address. Keep that terminal open, then open `http://localhost:9080` on your computer. For routine team access, the administrator must provide a shared HTTPS address and register its matching OAuth2 callback; this guide uses localhost for the initial setup.

## 9. Choose a discipline and configure your assistant

Choose **Architect**, **Structural engineer**, **MEP**, **Construction manager** or **LCFM**. Your profile belongs to your Discord account, even if another team member chooses the same discipline.

| Section | Use it for |
|---|---|
| Sonne & team | Shared assistant character, tasks, team values, owners and urgent requests. Click **River Team** or its pencil on the home screen to rename the team. |
| Project | The project brief, estimates, costs, source files and change history. |
| Discipline profile | Your introduction, goals, values, information and files. **Priorities** groups shared requests for the other disciplines. |

Signed-in drafts save automatically to the Docker installation. Select **Review prompt**, check the compiled instructions and choose **Apply to Sonne** to publish the shared brief and your personal instructions. Sonne uses them on your next permitted Discord request. Other project records, such as estimates, costs, owner feedback and shared files, update its project memory when saved.

**Check:** Apply reports success. Reloading the signed-in workspace retains your changes.

**Explore the workspace** opens a temporary preview. Its changes disappear on reload; it does not save to Docker or publish a prompt. For a missing sign-in button, check the OAuth2 Client Secret. For an invalid redirect, compare the full callback with step 5. An access denial means the signed-in account's User ID is not in the permitted list.

# Verify the installation

## 10. Test a Discord reply

In the `sonne` channel, use an account whose User ID you entered. Type `@Sonne` and **select the bot from Discord's suggestions** to create a real mention. Then send:

```
@Sonne Reply with exactly: sonne-ok
```

**Check:** A visible reply contains `sonne-ok`. The first CPU response may take several minutes; avoid sending duplicate requests while it is processing.

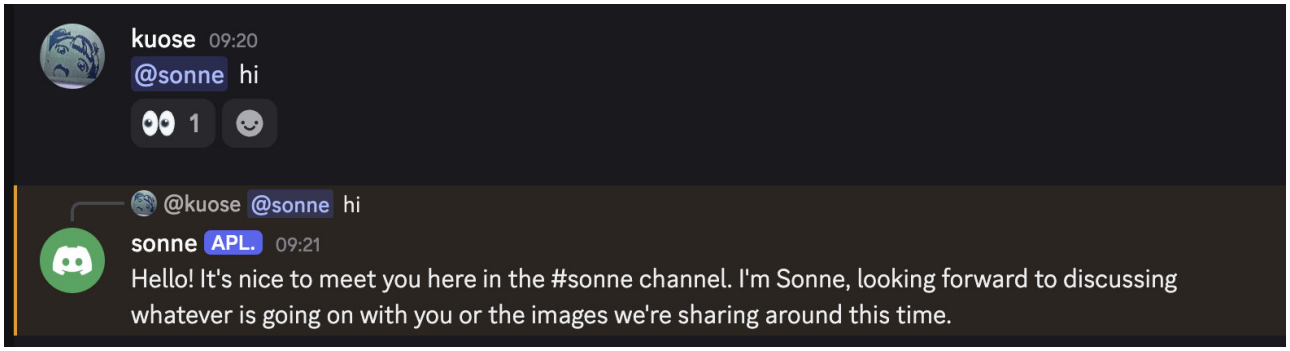



Figure 10. Ordinary message delivery observed during deployment. Use the prompts on this page for the acceptance checks.

## 11. Test an image and a saved file

In the Linux home folder, open **agents / sonne / examples**. Attach `vision-check.png` to a Discord message, mention the bot and ask it to read the large label and identify the red shape. If Discord runs on another computer, transfer the image there first.

**Check:** The response identifies **SONNE 42** and a **red square**. Verify both details; local models can misidentify image content.

Next, send the following message, using the same mention method:

```
@Sonne Create /workspace/files/smoke.txt containing exactly SONNE FILE OK.
Read it back.
```

In the **Linux terminal**, check the actual saved file:

```
cd ~/agents/sonne
cat workspace/files/smoke.txt
```

**Check:** The terminal prints `SONNE FILE OK`. If the file already exists, confirm the bot's overwrite request or use a new filename in both places.

**Recovery and troubleshooting**

| Symptom | Action |
|---|---|
| Download interrupted | Return to `~/sonne-installer` and run `bash scripts/install-sonne.sh` again. Do not repeat `git clone`. |
| Docker permission denied | Sign out of Linux completely and sign back in, or reconnect SSH, then retry the command. |
| Discord probe failed | Correct the token or IDs using `nano ~/agents/sonne/.env`. Save with **Ctrl+O**, Enter; exit with **Ctrl+X**, then rerun the installer. |
| No Discord reply | Check the real mention, permitted User ID, Channel ID, channel permissions and Message Content Intent. Then inspect status and logs below. |

Run these commands in the **Linux terminal**:

```
cd ~/agents/sonne
export COMPOSE_FILE=compose.yaml:compose.override.yaml:compose.prompt-studio.yaml
docker compose ps
docker compose logs --tail=100 openclaw prompt-studio
```

`ollama`, `openclaw` and `prompt-studio` should all be running and healthy. If a check still fails, retain the exact error message and the failed step for support. Remove tokens or other credentials before sharing diagnostic output.